

Übung 5: Zeitreihen, Schleifen, Gliederungsänderungen und Joins

Voraussetzungen

- RStudio-Projekt = projekt/. Datenzugriff relativ als data/....
- Pakete: dplyr, tidyr, purrr, readr (installiert).
- Startdatei: R/lab5_zeitreihen_joins.R (mit # TODO-Lücken).
- Erwartete Ergebnisse stehen am Ende dieses Übungsblatts.

Datensätze

Die Dateien enthalten die Spalten `jahr`, `quartal`, `bereich_code`, `bereich_name`, `wert` (Mio. EUR, jeweilige Preise). Es gibt 10 Wirtschaftsbereiche (A, BE, F, GI, J, K, L, MN, OQ, RU).

- data/lieferung_2024q4.csv (Vintage 1): 2018Q1 bis 2024Q4, 280 Zeilen.
- data/lieferung_2025q1.csv (Vintage 2): dieselbe Reihe, aber 2023/2024 leicht revidiert und ein neues Quartal 2025Q1, 290 Zeilen.
- data/klassifikation_korrespondenz.csv: alter Code zu neuem Code mit anteil. GI wird in G (0,52), H (0,36), I (0,12) gesplittet, alle anderen Bereiche 1:1.

Der Schlüssel für jede Beobachtung ist die Kombination `jahr + quartal + bereich_code`.

Schritt 0: Lieferungen einlesen

```
library(dplyr); library(tidyr); library(purrr); library(readr)

v1 <- read_csv("data/lieferung_2024q4.csv", show_col_types = FALSE)
v2 <- read_csv("data/lieferung_2025q1.csv", show_col_types = FALSE)
```

Schritt 1: Zeitachse, Wachstumsrate, Index

Bilden Sie einen fortlaufenden Quartalsindex `zeit_idx`, der die Quartale streng monoton ordnet (2018Q1 = 0, 2018Q2 = 1, ...). Er dient als verlässliche Sortier- und `lag()`-Grundlage. Berechnen Sie dann je Bereich:

- `wert_vorquartal = lag(wert)`,
- `wachstum_pct = (wert / wert_vorquartal - 1) * 100`,
- `index = wert / first(wert) * 100` (Basis = erstes Quartal der Reihe).

```
mit_zeit <- function(df) {
  df |>
  mutate(
    zeit_label = paste0(jahr, "Q", quartal),
    zeit_idx = NA_real_ # TODO: fortlaufenden Quartalsindex bilden
  )
}

v1_zeit <- mit_zeit(v1)
v1_raten <- v1_zeit # TODO: sortieren, gruppieren und Kennzahlen berechnen
```

Hinweis: `lag()` braucht eine korrekt sortierte Reihe innerhalb der Gruppe. Sortieren Sie immer vor `lag()`. Das erste Quartal je Bereich hat keinen Vorwert, dort ist `wachstum_pct` erwartungsgemäß NA.

Schritt 2: Schleifen, zweimal dasselbe Ergebnis

Berechnen Sie das Jahresmittel der Wachstumsrate je Bereich. Schreiben Sie es zuerst als klassische `for`-Schleife über die Bereiche und sammeln Sie die Teilergebnisse in einer Liste. Schreiben Sie es dann idiomatisch mit `purrr::map()` und `purrr::list_rbind()`. Beide Wege müssen identisch sein.

```
bereiche <- sort(unique(vl_raten$bereich_code))

jahresmittel_for <- list()
for (b in bereiche) {
  # TODO: Bereich filtern, Jahresmittel bilden und in der Liste ablegen.
}

jahresmittel_for <- NULL # TODO: Teilergebnisse verbinden und sortieren
jahresmittel_map <- NULL # TODO: dieselbe Berechnung mit map() und list_rbind()

# TODO: Beide Ergebnisse mit all.equal() prüfen.
```

Hinweis: `map()` ruft die Funktion je Element auf und liefert eine Liste. `list_rbind()` bindet die Ergebnis-Tibbles anschließend zeilenweise zusammen. Das ersetzt das manuelle `list()` plus `bind_rows()` der `for`-Variante.

Schritt 3: Aktualisierung ausgewählter Jahre

In der Praxis ersetzt eine neue Lieferung nicht alles, sondern revidiert ausgewählte Jahre und liefert neue Quartale. Ersetzen Sie in Vintage 1 die Werte der Jahre 2023 und 2024 durch die revidierten Werte aus Vintage 2 (per Schlüssel) und hängen Sie 2025Q1 an.

```
schluessel <- c("jahr", "quartal", "bereich_code")

revisionen <- NULL # TODO: revidierte Werte für 2023/2024 filtern
vl_aktualisiert <- vl # TODO: mit rows_update() aktualisieren
neu_2025q1 <- NULL # TODO: neues Quartal filtern

# TODO: neues Quartal anhängen und Ergebnis sortieren.
```

Hinweis: `rows_update()` ersetzt nur Zeilen, deren Schlüssel im Ziel bereits existieren. Neue Schlüssel (wie 2025Q1) sind dafür nicht geeignet, die hängen Sie mit `bind_rows()` an. Die Zeilenzahl steigt dadurch von 280 auf 290.

Schritt 4: Joins über die Schlüssel

```
vergleich_left <- NULL # TODO: left_join über schluessel
vergleich_inner <- NULL # TODO: inner_join über schluessel
nur_in_v2 <- NULL # TODO: anti_join von v2 gegen v1
```

- `left_join`: behält alle Vintage-1-Schlüssel (280) und ergänzt den Vintage-2-Wert.
- `inner_join`: nur gemeinsame Schlüssel. Da alle V1-Schlüssel auch in V2 stehen, ebenfalls 280.
- `anti_join`: die Schlüssel, die nur in Vintage 2 vorkommen. Das ist genau das neue Quartal 2025Q1, also 10 Zeilen (eine je Bereich).

Schritt 5: Gliederungsänderung GI in G/H/I

Die Korrespondenztabelle splittet GI über den `anteil` in G (0,52), H (0,36), I (0,12). Prüfen Sie zuerst, dass die Anteilssumme je alter Code genau 1 ergibt, sonst wäre der Split nicht summenerhaltend. Joinen Sie dann die Daten mit der Korrespondenztabelle und verteilen Sie `wert` proportional über `anteil`.

```
korresp <- read_csv("data/klassifikation_korrespondenz.csv", show_col_types = FALSE)

anteils_check <- NULL # TODO: Anteilssumme je alt_code berechnen
# TODO: Anteilssummen mit stopifnot() und Toleranz prüfen.

neu_gliederung <- NULL # TODO: Korrespondenz anwenden und Werte verteilen
```

Prüfen Sie den Summenerhalt mit Toleranz: je Quartal muss $G + H + I$ gleich dem alten GI-Wert sein.

```
gi_alt <- NULL # TODO: GI je Quartal aggregieren
ghi_neu <- NULL # TODO: G, H und I je Quartal aggregieren
split_check <- NULL # TODO: beide Summen verbinden und Abweichung berechnen

# TODO: Summenerhalt mit stopifnot() und Toleranz prüfen.
```

Stolperfalle: Vergleichen Sie Gleitkommazahlen nie mit `==`, sondern immer mit einer Toleranz (`abs(a - b) < 1e-6`). Der Anteil-Split erzeugt winzige Rundungsreste, die kein Fehler sind.

Schritt 6: Quartalssummen kontrollieren

Berechnen Sie zur Kontrolle die Summe aller Bereiche je Quartal und geben Sie die letzten Quartale aus. Das ist die übliche Aggregat-Plausibilisierung über alle Bereiche.

```
aggregat <- NULL # TODO: Summe aller Bereiche je Quartal berechnen

# TODO: letzte fünf Quartale ausgeben.
```

Ausführung und Kontrolle

Vervollständigen Sie `R/lab5_zeitreihen_joins.R` und führen Sie die Datei aus dem Projektordner mit `Rscript R/lab5_zeitreihen_joins.R` aus.

Kontrollwerte:

- Vintage 1 hat 280 Zeilen, Vintage 2 hat 290 Zeilen.
- `for`-Schleife und `map() |> list_rbind()` liefern ein identisches Jahresmittel-Ergebnis (Check bestanden).
- Nach Update + Anhang hat die aktualisierte Lieferung 290 Zeilen.
- `anti_join` findet genau 10 neue Schlüssel, alle aus dem Quartal 2025Q1.
- Nach der Gliederungsänderung gibt es 12 Bereiche (GI ersetzt durch G, H, I).
- Summenerhalt-Check $GI = G + H + I$ für alle Quartale bestanden (maximale Abweichung im Bereich $1e-11$).
- Gesamtsumme vor und nach der Reklassifikation unverändert.

Zusatzaufgaben (optional)

- Erweitern Sie den Vergleich um die absolute und relative Abweichung (`wert_v2 - wert_v1`) je Schlüssel und filtern Sie die Zeilen mit der größten Revision.
- Berechnen Sie zusätzlich eine Vorjahresrate (`lag(wert, 4)`) statt der Vorquartalsrate.